GALILEO Graph Databases Group

Implementation of GALOIS Features – Second Check

(Queries 18–30, Replication of Table 3 – Third Column)

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

December 2, 2025

Contents

1 Section dedicated to the missing features of previous deadline (SQL & NL baselines)

1.1 Inclusion of Missing Models (7B and B Variants)

1.2 Iterative System Workflow

1.3 Result Completion through Iterative Prompting

1.4 Explicit Comparison with Paper Results

1.5 Analysis of the Results

1.6 Naming Conventions and System Consistency

2 Introduction & Goals of GALOIS Implementation

As we saw in the previous task Large Language Models (LLMs) increasingly support structured data extraction using declarative queries such as Natural Language (NL) and SQL. However, when queries become more complex and require structured tabular outputs with filtering, joins, and aggregations, direct prompting of LLMs leads to inaccurate or incomplete results. As shown in the Galois paper, both NL and SQL prompting struggle with factual correctness, tuple completeness, and recall accuracy.

To address these limitations, the Galois system introduces a novel query execution framework that uses the LLM as a storage layer, while separating the execution pipeline into logical and physical operators, similar to traditional DBMSs. Instead of issuing a single LLM request, Galois decomposes SQL queries into a sequence of structured scan and filtering operations that improve recall, tuple completeness, and overall output consistency.

Scope of This Deliverable. In this checkpoint, we implement **Galois** framework to bridge declarative SQL querying with the latent knowledge bases of Large Language Models (LLMs), focusing on translating theoretical query execution paradigms, specifically physical scan strategies (Table Scan and Key Scan) and logical optimization techniques (push-ALL, push-SELECTIVE, push-WO) or merge all the optimizations (GALOIS F) into a modular Python-based pipeline integrated with DuckDB for robust schema management. The report details the system's orchestration mechanism, which manages the lifecycle from SQL parsing to dynamic prompt generation and iterative memory management for tuple deduplication. Furthermore, it analyzes the implementation of distinct Query Planning variants (for instance Push-All or Confidence-based execution), exploring the trade-offs between delegating filter evaluation to the generative model and performing relational post-processing locally (especially for JOIN operations).

3 GALOIS system architecture

This section describes the implementation of **GALOISWO** (Galois Without Optimizations), the baseline version of the Galois framework implemented for this checkpoint. Unlike the full Galois system, GaloisWO does not apply any logical or physical optimizations such as confidence-based pushdown, selective operator choice, metadata reasoning, or cost–quality trade-off. Instead, it adopts a simple two-stage execution pipeline:

1. **Key Extraction**: retrieve all possible primary keys of the target table using iterative prompting.
2. **Tuple Completion**: for each retrieved key, prompt the LLM to obtain remaining attribute values (one tuple at a time).

3.1 Key Retrieval Phase (First Stage)

3.2 Table-Scan Execution Phase (Second Stage)

In the Galois paper, two physical scan operators are introduced: *Table-Scan* and *Key-Scan*. While the full Galois system dynamically selects between them according to confidence and metadata, GaloisWO uses only **Table-Scan** in a fully deterministic manner, with no optimization or pushdown logic.

This behavior is clearly reflected in our implementation, specifically in the `table_scan()` method of the `GaloisExecutor` class. The execution shows the following key characteristics:

- **No condition pushdown:** The executor uses the original SQL query, but does not apply any optimization selection. The WHERE clause is either used as-is or fully omitted if not available. There is no selection of pushable conditions based on metadata or confidence values.
- **Schema-aware attribute discovery:** The executor retrieves full table attribute lists using the `GaloisWOSchemaManager`, which is a thin wrapper around the standard schema manager but isolates the GaloisWO namespace.

- **First Prompt and Iterative Prompt Generation:** The system constructs the first and iterative human prompts using: `build_table_scan_first_prompt()` and `build_table_scan_iter_prompt()` from `galois_prompts.py`.

  These prompts enforce:

  - valid JSON-only output,
  - table-aware attribute listing,
  - a maximum of 20 tuples per iteration,
  - and no natural language explanations.

- **Iterative Expansion:** The executor issues multiple LLM calls (up to `max_iter`, default 4), collects new tuples, and stops when no new data is returned.

- **Duplicate Removal and JSON Recovery:** The system hashes tuple contents (via `normalise_row()`) to track and deduplicate results across iterations. It also attempts to recover valid JSON even from partially truncated or malformed LLM output.

Unlike Key-Scan, which explicitly extracts primary keys first and then populates attributes one by one, Table-Scan retrieves complete tuples directly using LLM in multiple iterations.

3.3 Table vs Key-Scan in GaloisWO

4 Logical Optimization (Query PLanning)

The Logical Optimization phase determines what information needs to be extracted from the LLM and which filtering conditions should be "pushed down" to the generative model versus applied locally. This decision-making process is critical to balancing the trade-off between retrieval precision and the risk of hallucination.

4.1 Predicate Push-Down Strategy

The system implements a flexible planning mechanism capable of generating different execution plans based on the configured optimization level. The planner analyzes the **WHERE** clause of the parsed SQL query and partitions the conditions into two sets: **conditions_to_push** (integrated into the prompt) and **residual_conditions** (deferred to the post-processor). The framework supports four distinct logical variants:

- **No-Push (GALOIS-WO)**: Treats the LLM purely as a data source. No filtering conditions are included in the prompt; the system retrieves the entire extension of the table (or its keys) and performs all filtering locally.

- **Push-All (GALOIS-F)**: Maximizes delegation by translating all SQL predicates into natural language constraints within the prompt. This assumes high model capability in understanding and applying complex logic.

- **Push-Selective (GALOIS-S)**: A heuristic approach that identifies and pushes only the most selective condition, using the LLM's confidence self-assessment as a proxy to determine which conditions are "safe" or "selective" enough to be handled directly by the generative model and aims to reduce the search space without overwhelming the model context.
- **Push-Confident(GALOIS-F)**: An adaptive strategy that utilizes a **ConfidenceEstimator**. The system queries the LLM to self-evaluate its confidence in correctly assessing specific predicates (returning a "HIGH" or "LOW" score). Only conditions with high confidence scores are pushed down, while ambiguous ones are retained for local execution.

4.2 Join Handling

For multi-table queries, the logical planner decomposes the join operation into independent scan tasks. The system identifies all tables involved in the query (FROM clause and JOIN clauses) and generates a sub-plan for each. The planner ensures that alias references in the SQL query are correctly mapped to their respective table scans, allowing the Executor to fetch data for each relation independently before they are rejoined in the Post-Processing phase.

5 Physical Optimization (Execution Strategies)

While Logical Optimization defines the scope of the query, Physical Optimization defines the algorithmic strategy for data retrieval. Our system implements iterative algorithms designed to overcome the context window limitations of LLMs and to ensure comprehensive data extraction.

5.1 Table-Scan Operator

!!!!TO DO!!!!

5.2 Key-Scan Operator

!!!!TO DO!!!!

5.3 Adaptive Strategy Selection

The framework includes an **auto** execution mode that dynamically selects between Table-Scan and Key-Scan. This decision is mediated by the **ConfidenceEstimator**, which evaluates the query complexity and the number of requested columns. The estimator propagates a confidence score (τ) based on the model's self-assessment; if the confidence is below a specific threshold or if the schema is particularly wide, the system defaults to the Key-Scan strategy to ensure data integrity, otherwise, it proceeds with the more direct Table-Scan. This handling mode is useful for GALOIS-F logical plan.

6 Execution Cycle & Memory Handling

The interaction between the deterministic orchestration layer and the generative model is governed by a stateful execution cycle. Unlike standard single-turn LLM interactions, the Galois framework maintains a persistence layer across iterations to maximize recall and prevent redundancy. This logic is encapsulated within the **GaloisExecutor** class and its specialized internal memory component, **GaloisMemory**.

6.1 Stateful Context Management

!!!!EXPLAIN OF THE GALOISMEMORY OBJECT AND ITS FUNCTIONS !!!!!!

6.2 The Iterative execution loop

!!!!EXPLAIN THE LOOP FOR RETRIEVAL PROCESS (MAX_ITERATIONS ECC...)!!!!!

6.3 Termination Logic and Convergence

!!!!EXPLAIJN UNDER WHICH CONDITION THE LOOP TERMINATES !!!!

7 Post-Processing

The Post-Processing module serves as the deterministic convergence point of the architecture, addressing the inherent impedance mismatch between the probabilistic, unstructured output of the Large Language Model and the strict typing requirements of the SQL standard. This phase transforms the raw tuple sets retrieved by the Executor into a consistent relational state, enabling the execution of complex logical operations that cannot be reliably offloaded to the generative model.

7.1 Data Aggregation and Virtualization

Upon the completion of the physical scan phase (whether Table-Scan or Key-Scan), the retrieved tuples are aggregated into a local staging structure, referred to as the **"Data Lake"** which maps table names to lists of dictionaries. To bridge the gap to a relational environment, the system instantiates an ephemeral, in-memory DuckDB connection. The aggregated data is converted into Pandas DataFrames and registered as virtual views within this database instance, effectively creating a transient relational schema populated by the LLM-generated data.

7.2 Heuristic Type Inference and Sanitization

A critical component of this phase is the "Smart Auto-Casting" mechanism implemented within the Galois orchestrator. Since LLMs typically generate JSON values as strings (e.g., "1000", "N/A"), direct SQL operations often fail due to type mismatches. The system iterates through the columns of the intermediate DataFrames, attempting to coerce values into numeric types using **pd.to_numeric**.

To prevent erroneous casting (e.g., converting legitimate alphanumeric identifiers into NaN), the algorithm employs a statistical threshold: if the coercion results in a null

rate exceeding 95% (indicating that the column is likely textual), the transformation is reverted, and the column is treated as a string. Furthermore, string attributes undergo sanitization procedures, such as whitespace stripping, to ensure that join keys match correctly (e.g., resolving discrepancies between "Arizona " and "Arizona").

7.3 Deterministic Query Execution

Once the data is virtualized and typed, the system executes the logical query plan locally. This step involves running the original SQL query (sanitized of specific schema prefixes like **.target**) against the virtual tables. This local execution performs two vital functions:

- **Join Reconstruction**: It re-establishes the relational links between disparate datasets that were retrieved via independent scan operations, ensuring the correct implementation of INNER or LEFT joins defined in the original AST.
- **Residual Filtering**: It applies the **residual_conditions** predicates identified during the Logical Optimization phase as too complex or numerically precise for the LLM (e.g., strict inequalities like length_in_km > 750). This ensures that the final result set adheres strictly to the logical constraints of the user's query, filtering out any hallucinations or imprecise data artifacts introduced by the model.

8 Numerical Results

In this section we will shoew a comparison between the metrics of original GALOIS paper written by the authors, and the metrics that we obtained from our GALOIS implementation.

8.1 GALOIS WO Results comparison

Metric	PAPER_Galois WO	GaloisWO
F1-Cell	0.518	0.088
Cardinality	0.691	0.265
Tuple Constr.	0.389	0.048
AVG-Score	0.531	0.134
#Tokens (M)	19.710	
Avg Time	1460.000	

Table 1: GaloisWO comparison

Metric	PAPER_Galois S	GaloisS
F1-Cell	0.480	0.434
Cardinality	0.655	0.667
Tuple Constr.	0.365	0.297
AVG-Score	0.500	0.466
#Tokens (M)	0.960	
Avg Time	130.000	

Table 2: GaloisS comparison

Metric	PAPER_Galois A	GaloisA
F1-Cell	0.543	0.417
Cardinality	0.799	0.614
Tuple Constr.	0.448	0.279
AVG-Score	0.592	0.436
#Tokens (M)	0.950	
Avg Time	120.500	

Table 3: GaloisA comparison

Metric	PAPER_Galois F	GaloisF
F1-Cell	0.563	0.319
Cardinality	0.835	0.528
Tuple Constr.	0.464	0.166
AVG-Score	0.622	0.338
#Tokens (M)	1.720	
Avg Time	47.400	

Table 4: GaloisF comparison

8.2 GALOIS S Results comparison

8.3 GALOIS A Results comparison

8.4 GALOIS F Results comparison

9 Implementative challenges encountered

The most challenging aspect of the implementation was the development and the results enhancing for the **Key Scan** physical operator, in particular we obtained very poor performance in `GALOIS F` and `GALOIS WO` logical plans. This is mainly due to the fact in the concept of Key Scan we first ask to the LLM to retrieve data using only the primary key for each table, so fo instance if we have the `venezuela_presidents` table with `president_name` defined as table's Primary Key, in the first phase we will ask to the LLM to retrieve the presidents of all word and only in the second phase we will apply the original query to the result set returned by the LLM; but in principle if the LLM didn't return any president from Venezuela the final result set post query filtering will be empty. This behavior is common to all datasets used in the `GALOIS F` and `GALOIS WO` versions, so we had to find a way to improve the Key Scan's retrieval process effectiveness.

Another issue we faced was managing DuckDB connections during execution, since several classes open connections to retrieve column data types or, more broadly, the database schema. For address this problem, the system simply closes the connection to the database when the actions for that specific connection are completed, thus avoiding having too many open connections at the same time.

!!!!NOMINARE IL FATTO DI POTER OTTIMIZZARE LA STRUTTURA E LA SEMANTICA DEI PROMPT, VISTO CHE QUELLI USATO DA GALOIS NON SONO EFFICACI !!!!

10 Contributions

Alessio Demo (Badge number: 2142885)

•

Francesco Pivotto (Badge number: 2158296)

•

Giorgia Amato (Badge number: 2159999)

•